Security baseline

This describes the controls in the code today, what must be added before launch, and how to report a vulnerability. It also serves as the "documented information security programme" expected under the IT Act SPDI Rules (chapter 3).

1. Controls already implemented

Area	Control	Where
Authentication	Passwordless OTP; codes stored hashed; attempt limits; expiry; per-identifier and per-IP rate limits	`modules/auth/otp.ts`, `shared/lib/guard.ts`
Sessions	Random 256-bit token in an HTTP-only cookie; only a SHA-256 hash stored; revocable; 30-day expiry; deactivated users lose sessions	`modules/auth/session.ts`, `modules/admin/actions.ts`
Authorisation	Role checks in layouts **and** inside every server action (`assertRole`); resource ownership checks (e.g. a customer can only pay their own booking)	`modules/auth/guards.ts`, module actions
Input validation	Zod schemas on every action and settings group	`modules/*/service.ts`, `modules/settings/schemas.ts`
Output safety	Markdown renderer escapes HTML by construction; React escaping elsewhere	`shared/lib/markdown.ts`
Transport and headers	HSTS (preload), CSP, `X-Frame-Options: DENY`, `frame-ancestors 'none'`, nosniff, strict referrer, restrictive Permissions-Policy	`next.config.ts`
Webhooks	Razorpay signatures verified with constant-time comparison; cron protected by bearer secret	`modules/payments/service.ts`, `api/cron/tick`
Concurrency	Atomic conditional updates for offer acceptance and status changes	`modules/dispatch/service.ts`
Audit	Staff changes logged with actor, entity and IP	`AuditLog`
Privacy	No PII in analytics; staff excluded from analytics; OTPs redacted in notification log	`modules/analytics/track.ts`
Containers	Non-root standalone image with healthcheck	`Dockerfile`

2. Must add before launch

1. **Error monitoring and alerting** (Sentry or similar), with PII scrubbing.
2. **Log retention for 180 days** in storage we control (CERT-In Directions), with access limited to engineering leads.
3. **Secrets:** strong `AUTH_SECRET` (≥ 32 random bytes) and `CRON_SECRET`; `EXPOSE_DEV_OTP` unset; secrets only in the hosting provider's encrypted env vars; a rotation procedure. Rotating `AUTH_SECRET` signs everyone out.
4. **Least privilege:** only the super admin can grant ADMIN; dispatchers cannot view bank details (review admin pages); MFA on Vercel, Render, Neon, GitHub, Razorpay and email-provider accounts.

5. **Backups:** Neon point-in-time restore enabled; quarterly restore drill.
6. **Dependency hygiene:** `npm audit` in CI; Dependabot or Renovate.
7. **CSP hardening:** move from `'unsafe-inline'` scripts to nonces.
8. **File uploads (when KYC/photos ship):** private bucket, signed short-lived URLs, type and size checks, and malware scanning for documents.

3. Security testing

- Run an authenticated DAST scan (e.g. StackHawk/ZAP) against a staging deployment before launch and after major changes.
- Manual checks: IDOR on `/invoice/[code]` and `/track/[code]` (codes are random and unguessable; confirm no PII leaks beyond what the customer needs), privilege escalation via server actions, rate-limit bypass on OTP endpoints.

4. Incident response

Follow `../operations/INCIDENT-RESPONSE.md` (docs/operations/INCIDENT-RESPONSE.md). Specified incidents must be reported to CERT-In **within 6 hours** of noticing them.

5. Reporting a vulnerability

Email the security contact listed on the website (Admin → Settings → Contact → support email, until a dedicated `security@` address exists). Please include steps to reproduce. We aim to acknowledge within 2 business days. Do not access other users' data or degrade the service while testing.